

Python OOP

Object-Oriented Programming · Complete Cheat Sheet · Root to Advanced

1. WHAT IS OOP?

OOP (Object-Oriented Programming) is a way to structure code using objects — bundles of data (attributes) and behavior (methods). Everything in Python is an object.

- ✓ Classes = Blueprints | Objects = Actual things built from the blueprint
- ✓ 4 Pillars: Encapsulation · Inheritance · Polymorphism · Abstraction
- ✓ Makes code modular, reusable, and easy to maintain

Keyword / Syntax	What It Means
Class	Blueprint / template (e.g. Car design)
Object	Instance of a class (e.g. your actual car)
Attribute	Data stored in the object (e.g. car.color)
Method	Function defined inside a class (e.g. car.start())
self	Reference to the current object instance

2. CLASS & OBJECT — THE BASICS

► Defining a Class and Creating Objects

EXAMPLE

```
class Dog:                                # class keyword + PascalCase name
    species = 'Canis lupus'                # Class attribute (shared by all)

    def __init__(self, name, age):         # Constructor – runs on object creation
        self.name = name                   # Instance attribute (unique per object)
        self.age = age

    def bark(self):                         # Instance method
        return f'{self.name} says: Woof!'

    def info(self):
        return f'{self.name}, Age: {self.age}, Species: {Dog.species}'

# Creating objects (instances)
dog1 = Dog('Bruno', 3)
dog2 = Dog('Luna', 5)
```

```
print(dog1.bark())
print(dog2.info())
print(Dog.species)  # access via class name
```

```
OUT      Bruno says: Woof!
         Luna, Age: 5, Species: Canis lupus
         Canis lupus
```

🔗 self is always the first parameter of every instance method — it refers to the calling object.

🔗 __init__ is NOT the constructor itself; Python calls __new__ first, then __init__ to initialize.

🔗 3. ATTRIBUTES IN DEPTH

► Class Attribute vs Instance Attribute

EXAMPLE

```
class Counter:
    count = 0          # Class attribute – shared by ALL instances

    def __init__(self, name):
        Counter.count += 1 # modify class attribute
        self.name = name   # Instance attribute – unique per object

a = Counter('A')
b = Counter('B')
c = Counter('C')

print(Counter.count)  # 3 (class attribute)
print(a.name)         # A (instance attribute)
print(b.name)         # B
```

```
OUT      3
         A
         B
```

🔗 If you assign to self.class_attr, you create a new instance attribute — not modifying the class attribute.

► Built-in Attribute Utilities

BUILT-IN FUNCTIONS

```
class Car:
    def __init__(self, brand, speed):
        self.brand = brand
        self.speed = speed

c = Car('Toyota', 180)
```

```
print(hasattr(c, 'brand'))    # True - does attribute exist?
print(getattr(c, 'brand'))    # Toyota - get attribute value
setattr(c, 'color', 'Red')    # add/change attribute dynamically
print(c.color)                # Red
delattr(c, 'color')           # delete attribute
print(vars(c))                # {'brand': 'Toyota', 'speed': 180}
```

OUT

```
True
Toyota
Red
{'brand': 'Toyota', 'speed': 180}
```

4. METHODS — ALL TYPES

► 4a. Instance Method (most common)

Operates on the object. First parameter is always self.

EXAMPLE

```
class Circle:
    pi = 3.14159

    def __init__(self, radius):
        self.radius = radius

    def area(self):          # instance method
        return self.pi * self.radius ** 2

c = Circle(5)
print(c.area())           # 78.53975
```

OUT

78.53975

► 4b. Class Method — @classmethod

Operates on the CLASS, not instance. First parameter is cls. Used as alternative constructors.

EXAMPLE

```
class Person:
    population = 0

    def __init__(self, name, age):
        self.name = name
        self.age = age
        Person.population += 1

    @classmethod
    def get_population(cls):          # cls = Person class itself
        return cls.population

    @classmethod
    def from_birth_year(cls, name, birth_year): # alt constructor
        age = 2025 - birth_year
        return cls(name, age)         # creates a Person object

p1 = Person('Aman', 28)
p2 = Person.from_birth_year('Priya', 1998)

print(Person.get_population())      # 2
print(p2.name, p2.age)              # Priya 27
```

OUT

2
Priya 27

► 4c. Static Method — @staticmethod

No access to class or instance. A utility function logically grouped inside a class.

EXAMPLE

```
class MathHelper:
    @staticmethod
    def is_even(n):          # no self or cls
        return n % 2 == 0

    @staticmethod
    def clamp(value, min_v, max_v):
        return max(min_v, min(value, max_v))

print(MathHelper.is_even(4))      # True
print(MathHelper.clamp(150, 0, 100)) # 100
```

OUT

True
100

Keyword / Syntax	What It Means
Instance method	def method(self) — works on the object
Class method	@classmethod def m(cls) — works on the class
Static method	@staticmethod def m() — utility, no self/cls

5. ENCAPSULATION — HIDING DATA

Encapsulation = bundling data + methods, and restricting direct access to some components. Python uses naming conventions (not strict enforcement).

Keyword / Syntax	What It Means
<code>self.name</code>	Public — accessible anywhere (default)
<code>self._name</code>	Protected — convention: 'internal use', still accessible
<code>self.__name</code>	Private — name-mangled to <code>_ClassName__name</code> , harder to access

EXAMPLE

```
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner          # public
        self._bank = 'HDFC'        # protected
        self.__balance = balance    # private (mangled)

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):           # controlled access
        return self.__balance

acc = BankAccount('Nishant', 10000)
acc.deposit(5000)
print(acc.get_balance())           # 15000
# print(acc.__balance)             # AttributeError!
print(acc._BankAccount__balance)   # 15000 (name mangling)
```

OUT

15000
15000

► Properties — @property (Pythonic Getters/Setters)

Instead of `get_x()` / `set_x()` methods, use `@property` for clean attribute-style access with validation.

EXAMPLE

```
class Temperature:
    def __init__(self, celsius):
        self._celsius = celsius

    @property
    def celsius(self):               # getter
        return self._celsius

    @celsius.setter
    def celsius(self, value):        # setter with validation
        if value < -273.15:
            raise ValueError('Below absolute zero!')
        self._celsius = value
```

```
@property
def fahrenheit(self):          # computed property (no setter)
    return self._celsius * 9/5 + 32

t = Temperature(100)
print(t.celsius)               # 100   (looks like attribute, runs getter)
print(t.fahrenheit)           # 212.0
t.celsius = 25                 # runs setter
print(t.celsius)               # 25
```

OUT

```
100
212.0
25
```

? @property lets you add validation/logic while keeping the dot-notation interface clean.

6. INHERITANCE

A child class inherits attributes and methods from a parent class. Avoids code duplication.

► 6a. Single Inheritance

EXAMPLE

```
class Animal:                                # Parent class
    def __init__(self, name):
        self.name = name

    def speak(self):
        return f'{self.name} makes a sound.'

    def breathe(self):
        return f'{self.name} breathes air.'

class Dog(Animal):                            # Child class inherits Animal
    def speak(self):                        # Override parent method
        return f'{self.name} says: Woof!'

class Cat(Animal):
    def speak(self):
        return f'{self.name} says: Meow!'

d = Dog('Bruno')
c = Cat('Whiskers')

print(d.speak())    # overridden
print(d.breathe())  # inherited from Animal
print(c.speak())
```

OUT

```
Bruno says: Woof!
Bruno breathes air.
Whiskers says: Meow!
```

► 6b. super() — Calling Parent's Method

Use super() to call the parent class's method without hardcoding the parent class name.

EXAMPLE

```
class Vehicle:
    def __init__(self, brand, speed):
        self.brand = brand
        self.speed = speed

    def info(self):
        return f'{self.brand} – {self.speed} km/h'

class ElectricCar(Vehicle):
    def __init__(self, brand, speed, battery_kw):
        super().__init__(brand, speed)    # call parent __init__
        self.battery_kw = battery_kw      # add extra attribute
```



```

def info(self):                # extend parent method
    base = super().info()      # call parent info()
    return f'{base} | Battery: {self.battery_kw}kW'

tesla = ElectricCar('Tesla', 250, 100)
print(tesla.info())

```

OUT Tesla – 250 km/h | Battery: 100kW

► 6c. Multiple Inheritance & MRO

Python allows a class to inherit from multiple parents. MRO (Method Resolution Order) defines which parent's method is used. Check with `ClassName.__mro__`.

EXAMPLE

```

class Flyable:
    def move(self): return 'Flying'

class Swimmable:
    def move(self): return 'Swimming'

class Duck(Flyable, Swimmable): # Multiple inheritance
    pass

d = Duck()
print(d.move())                # 'Flying' – Flyable comes first in MRO
print(Duck.__mro__)            # shows MRO chain

```

OUT Flying
(<class 'Duck'>, <class 'Flyable'>, <class 'Swimmable'>, <class 'object'>)

? MRO follows C3 linearization. Python searches left-to-right through parent classes.

► 6d. isinstance() and issubclass()

TYPE CHECKING

```

print(isinstance(d, Duck))      # True
print(isinstance(d, Flyable))   # True – checks inheritance too
print(issubclass(Duck, Flyable)) # True
print(issubclass(Duck, list))   # False

```

OUT
True
True
True
False

7. POLYMORPHISM

Poly = many, morph = form. Same interface, different behavior. One function/loop that works on different types of objects.

► 7a. Method Overriding (Runtime Polymorphism)

EXAMPLE

```
class Shape:
    def area(self): return 0

class Rectangle(Shape):
    def __init__(self, w, h): self.w, self.h = w, h
    def area(self): return self.w * self.h

class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14 * self.r ** 2

class Triangle(Shape):
    def __init__(self, b, h): self.b, self.h = b, h
    def area(self): return 0.5 * self.b * self.h

shapes = [Rectangle(4,5), Circle(3), Triangle(6,8)]

for shape in shapes:
    # same call, different result
    print(f'{shape.__class__.__name__}: area = {shape.area()}')
```

OUT

```
Rectangle: area = 20
Circle: area = 28.26
Triangle: area = 24.0
```

► 7b. Duck Typing — 'If it quacks like a duck...'

Python doesn't check the TYPE — it checks if the object has the METHOD. If it has .speak(), we call it.

EXAMPLE

```
class Dog:
    def speak(self): return 'Woof!'

class Cat:
    def speak(self): return 'Meow!'

class Robot:
    def speak(self): return 'Beep boop!'

def make_speak(animal):
    # no type check needed
    print(animal.speak())

for obj in [Dog(), Cat(), Robot()]:
    make_speak(obj)
```

OUT

```
Woof!
Meow!
Beep boop!
```

🔗 Duck typing = focus on what an object CAN DO, not what it IS. Python's idiomatic style.

🔗 8. ABSTRACTION — ABC & ABSTRACT METHODS

Abstraction = hide complex implementation, show only what's necessary. Use ABC (Abstract Base Class) to force child classes to implement required methods.

EXAMPLE

```
from abc import ABC, abstractmethod

class Payment(ABC):          # Abstract Base Class
    @abstractmethod
    def pay(self, amount):    # abstract method – MUST be overridden
        pass

    @abstractmethod
    def refund(self, amount):
        pass

    def receipt(self, amount): # concrete method – shared logic
        return f'Receipt: Rs.{amount} paid via {self.__class__.__name__}'

class CreditCard(Payment):
    def pay(self, amount):
        return f'Paid Rs.{amount} via Credit Card'
    def refund(self, amount):
        return f'Refunded Rs.{amount} to Credit Card'

class UPI(Payment):
    def pay(self, amount):
        return f'Paid Rs.{amount} via UPI'
    def refund(self, amount):
        return f'Refunded Rs.{amount} to UPI'

# Payment() # TypeError: Can't instantiate abstract class
cc = CreditCard()
up = UPI()
print(cc.pay(500))
print(cc.receipt(500))
print(up.pay(200))
```

OUT

```
Paid Rs.500 via Credit Card
Receipt: Rs.500 paid via CreditCard
Paid Rs.200 via UPI
```

🔗 You CANNOT instantiate an abstract class directly. It enforces a 'contract' on all subclasses.

✦9. DUNDER (MAGIC) METHODS

Double underscore methods (`__init__`, `__str__`, etc.) let you customize how objects behave with built-in Python operations like `print()`, `+`, `len()`, `==`, and more.

► Common Dunder Methods Reference

Keyword / Syntax	What It Means
<code>__init__(self, ...)</code>	Constructor — called on object creation
<code>__str__(self)</code>	Returns string for <code>print()</code> / <code>str()</code> — human readable
<code>__repr__(self)</code>	Returns string for <code>repr()</code> — developer / debugging
<code>__len__(self)</code>	Returns value for <code>len(obj)</code>
<code>__add__(self, o)</code>	Defines <code>obj + obj</code> behaviour
<code>__sub__(self, o)</code>	Defines <code>obj - obj</code>
<code>__mul__(self, o)</code>	Defines <code>obj * obj</code>
<code>__eq__(self, o)</code>	Defines <code>obj == obj</code>
<code>__lt__(self, o)</code>	Defines <code>obj < obj</code>
<code>__contains__(self, item)</code>	Defines <code>'item in obj'</code>
<code>__getitem__(self, key)</code>	Defines <code>obj[key]</code>
<code>__setitem__(self, key, v)</code>	Defines <code>obj[key] = value</code>
<code>__iter__(self)</code>	Makes object iterable (used in for loops)
<code>__next__(self)</code>	Returns next item in iteration
<code>__call__(self, ...)</code>	Makes object callable like a function: <code>obj()</code>
<code>__del__(self)</code>	Destructor — called when object is garbage collected

► Practical Dunder Example — Vector Class

EXAMPLE

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        # print(v) → '(3, 4)'
        return f'({self.x}, {self.y})'

    def __repr__(self):
        # repr(v) → 'Vector(3, 4)'
        return f'Vector({self.x}, {self.y})'

    def __add__(self, other):
        # v1 + v2
        return Vector(self.x + other.x, self.y + other.y)

    def __eq__(self, other):
        # v1 == v2
        return self.x == other.x and self.y == other.y
```

```
def __len__(self):          # len(v) → int magnitude
    return int((self.x**2 + self.y**2) ** 0.5)
```

```
v1 = Vector(3, 4)
v2 = Vector(1, 2)
```

```
print(v1)          # (3, 4) – uses __str__
print(repr(v1))    # Vector(3, 4) – uses __repr__
print(v1 + v2)     # (4, 6) – uses __add__
print(v1 == v2)    # False – uses __eq__
print(len(v1))     # 5 – uses __len__
```

OUT

```
(3, 4)
Vector(3, 4)
(4, 6)
False
5
```

► __call__ — Making Objects Callable

EXAMPLE

```
class Multiplier:
    def __init__(self, factor):
        self.factor = factor

    def __call__(self, n):          # obj() syntax
        return n * self.factor

double = Multiplier(2)
triple = Multiplier(3)

print(double(10))  # 20 – calling the object like a function!
print(triple(7))  # 21
```

OUT

```
20
21
```

10. CLASS RELATIONSHIPS

► Composition — 'HAS-A' relationship

One class contains an instance of another as an attribute. Preferred over inheritance when the relationship is 'has-a' not 'is-a'.

EXAMPLE (Composition)

```
class Engine:
    def __init__(self, horsepower):
        self.hp = horsepower
    def start(self):
        return f'Engine ({self.hp}HP) started!'

class Car:
    def __init__(self, brand, hp):
        self.brand = brand
        self.engine = Engine(hp)      # Car HAS-A Engine

    def drive(self):
        return f'{self.brand}: {self.engine.start()} Let us go!'

my_car = Car('BMW', 300)
print(my_car.drive())
```

OUT BMW: Engine (300HP) started! Let us go!

📌 Prefer Composition over Inheritance when possible — it's more flexible and avoids deep hierarchies.

► Aggregation — 'USES-A' relationship

Objects are created independently and passed in. The contained object can exist without the container.

EXAMPLE (Aggregation)

```
class Player:
    def __init__(self, name):
        self.name = name

class Team:
    def __init__(self, team_name):
        self.team_name = team_name
        self.players = []      # aggregation

    def add_player(self, player):    # player exists independently
        self.players.append(player)

    def show(self):
        names = [p.name for p in self.players]
        print(f'{self.team_name}: {names}')

p1 = Player('Rohit')
p2 = Player('Virat')
p3 = Player('Dhoni')
```

```
team = Team('India XI')  
team.add_player(p1)  
team.add_player(p2)  
team.add_player(p3)  
team.show()
```

OUT

India XI: ['Rohit', 'Virat', 'Dhoni']

11. DATACLASSES (Python 3.7+)

@dataclass automatically generates __init__, __repr__, __eq__ etc. Perfect for classes that mainly store data.

EXAMPLE

```
from dataclasses import dataclass, field

@dataclass
class Student:
    name: str
    age: int
    grade: float = 0.0          # default value
    courses: list = field(default_factory=list) # mutable default

s1 = Student('Aman', 21, 8.5)
s2 = Student('Priya', 22)

print(s1)          # auto __repr__
print(s1 == s2)     # auto __eq__
s2.courses.append('Python')
print(s2)
```

```
OUT Student(name='Aman', age=21, grade=8.5, courses=[])
      False
      Student(name='Priya', age=22, grade=0.0, courses=['Python'])
```

Use field(default_factory=list) for mutable defaults — avoids the shared-list bug.

► Frozen Dataclass — Immutable objects

FROZEN DATACLASS

```
@dataclass(frozen=True)          # makes instances immutable
class Point:
    x: float
    y: float

p = Point(3.0, 4.0)
print(p)                        # Point(x=3.0, y=4.0)
# p.x = 10                      # FrozenInstanceError!
print(hash(p))                  # hashable — can be used in sets/dicts
```

```
OUT Point(x=3.0, y=4.0)
      3713081631934410656
```

12. ITERATORS & DESCRIPTORS

► Custom Iterator using __iter__ and __next__

EXAMPLE

```
class Countdown:
```



```

def __init__(self, start):
    self.start = start

def __iter__(self):
    # makes object iterable
    self.current = self.start
    return self

def __next__(self):
    # returns next value
    if self.current <= 0:
        raise StopIteration # signals end of iteration
    val = self.current
    self.current -= 1
    return val

for n in Countdown(5):
    print(n, end=' ')

```

OUT

5 4 3 2 1

► Descriptor Protocol — `__get__`, `__set__`, `__delete__`

Descriptors control attribute access at the class level. `@property` uses descriptors under the hood.

ADVANCED EXAMPLE

```

class PositiveNumber:
    def __set_name__(self, owner, name):
        self.name = name

    def __get__(self, obj, objtype=None):
        if obj is None: return self
        return obj.__dict__.get(self.name, 0)

    def __set__(self, obj, value):
        if value < 0:
            raise ValueError(f'{self.name} must be >= 0')
        obj.__dict__[self.name] = value

class Product:
    price = PositiveNumber() # descriptor instance
    quantity = PositiveNumber()

    def __init__(self, name, price, qty):
        self.name = name
        self.price = price # triggers __set__
        self.quantity = qty

p = Product('Laptop', 50000, 10)
print(p.price) # 50000
# p.price = -100 # ValueError: price must be >= 0

```

OUT

50000

13. __slots__ — MEMORY OPTIMIZATION

By default, each object has a `__dict__` (a dictionary) storing its attributes. `__slots__` replaces this with a fixed array — faster access and less memory.

EXAMPLE

```
class WithoutSlots:
    def __init__(self, x, y):
        self.x = x
        self.y = y

class WithSlots:
    __slots__ = ['x', 'y'] # declare allowed attributes
    def __init__(self, x, y):
        self.x = x
        self.y = y

import sys
a = WithoutSlots(1, 2)
b = WithSlots(1, 2)

print(sys.getsizeof(a.__dict__)) # ~232 bytes
# b has no __dict__ — uses slot array
# b.z = 3 # AttributeError — can't add new attrs
print(b.x, b.y) # 1 2
```

OUT

~232 bytes
1 2

Use `__slots__` when creating millions of small objects (e.g. in simulations/data pipelines). Typical savings: 40-50% memory.

14. QUICK REFERENCE — ALL OOP CONCEPTS AT A GLANCE

Concept	Syntax / Keyword	Key Point
Class definition	<code>class MyClass:</code>	PascalCase naming
Constructor	<code>__init__(self, ...)</code>	Runs on object creation
Instance attribute	<code>self.name = value</code>	Unique per object
Class attribute	<code>name = value</code> (class level)	Shared by all instances
Instance method	<code>def method(self)</code>	Operates on object
Class method	<code>@classmethod def m(cls)</code>	Operates on the class
Static method	<code>@staticmethod def m()</code>	Utility, no self/cls
Public attribute	<code>self.name</code>	Accessible anywhere
Protected attribute	<code>self._name</code>	Convention only
Private attribute	<code>self.__name</code>	Name-mangled
Property getter	<code>@property</code>	Attribute-style access

Property setter	@name.setter	Add validation on set
Inheritance	class Child(Parent):	IS-A relationship
super()	super().method()	Call parent method
Multiple inheritance	class C(A, B):	MRO resolves order
Override method	Redefine in child class	Runtime polymorphism
Abstract class	class X(ABC): @abstractmethod	Forces child to implement
Composition	self.engine = Engine()	HAS-A relationship
Aggregation	Pass obj as parameter	USES-A, independent life
__str__	print(obj) output	Human readable string
__repr__	repr(obj) / debug output	Developer debug string
__add__ etc.	obj + obj2	Operator overloading
__call__	obj(args)	Make obj callable
__iter__ / __next__	for x in obj	Custom iteration
dataclass	@dataclass decorator	Auto __init__, __repr__
__slots__	__slots__ = ['x','y']	Memory optimization
isinstance()	isinstance(obj, Class)	Check type + inheritance
issubclass()	issubclass(Child, Parent)	Check class hierarchy